CSC 452

FORMAL MODELS OF COMPUTATION

CMP 452Formal Models of Computation (3 Units) (L: 45)

Introduction; Automata theory: Roles of models in computation, Finite state Automata, Push-down Automata, Formal Grammars, Parsing, Relative powers of formal models. Basic computability: Turing-Machines, Universal Turing-Machines, Church's thesis, Solvability and Decidability.

Introduction

In theoretical computer science, the Theory of Computation (TOC) is the branch that deals with whether and how efficiently problems can be solved on a model of computation, using an algorithm. The field is divided into three major branches:

i. Automata theory,
ii. Computability theory and
iii. Computational complexity theory.

In order to perform a rigorous study of computation, computer scientists' work with a mathematical abstraction of computers called a model of computation. There are several models in use, but the most commonly examined is the Turing machine.

Why we need to study the Theory of Computation (TOC)

This lead to theory of what can be computed and what cannot. It involves sketch of some theoretical frameworks that can inform the design of programs to solve a wide variety of problems. Two major reasons are:

i. The shelf life of programming tools
ii. Applications of the theory are everywhere

i. The Shelf Life of Programming Tools

Implementations come and go. In the somewhat early days of computing, programming meant knowing how to write code like. Machine code and Assembly Code, then 1n 1957, Fortran appeared and made it possible for people to write programme that looked more straightforward like mathematics, then later Java changed the way we write program. Along the way, other programming languages became popular, at least within some circles.

Today's programmers can't read code from 50 years ago. Programmers from the early days could never have imagined what a program of today would look like. In the face of that kind of change, what does it mean to learn the science of computing?

The answer is that **there are mathematical properties**, both of problems and of algorithms for solving problems that depend on neither the details of today's technology nor the programming fashion *du jour (of the day in French)*. The theory will address some of those properties. Most of what we will discuss was known by the early 1970s (barely the middle ages of computing history). But it is still **useful in two key ways:**

1. It provides a set of abstract structures that are useful for solving certain classes of problems. These abstract structures can be implemented on whatever hardware/software platform is available.
2. It defines provable limits to what can be computed, regardless of processor speed or memory size. An understanding of these limits helps us to focus our design effort in areas in which it can pay off, rather than on the computing equivalent of the search for a perpetual motion machine.

The **goal** will be to discover fundamental properties of the problems themselves:

1. Is there any computational solution to the problem? If not, is there a restricted but useful variation of the problem for which a solution does exist?
2. If a solution exists, can it be implemented using some fixed amount of memory?

ii. Applications of the Theory Are Everywhere

Computers have revolutionized our world. They have changed the course of our daily lives, the way we do science, the way we entertain ourselves, the way that business is conducted, and the way we protect our security. Some of the application areas of TOC are:

- It enables both machine/machine and person/machine **communication**. Without them, none of today's applications of computing could exist. E.g. Network communication protocols are languages. Most Web pages are described using the Hypertext Markup Language, HTML. Music can be viewed as a language. And specialized languages enable composers to create new electronic music. Even very unlanguage-like things, such as sets of pictures, can be viewed as languages by, for example, associating each picture with the program that drew it.

- Both the design and the **implementation of modern programming languages** rely heavily on the theory of context-free languages. Context-free grammars are used to document the languages' syntax and they form the basis for the parsing techniques that all compilers use.

- People use **natural languages**, such as English, to communicate with each other. Since the advent of word processing, and then the Internet, we now type or speak our words to computers. So we would like to build programs to manage our words, check our grammar, search the World Wide Web, and translate from one language to another. Programs to do that also rely on the theory of context-free languages.

- Systems as diverse as **parity checkers**, vending machines (e.g. ATM), communication protocols, and building security devices can be straightforwardly described as finite state machines. E.g. A family of network communication protocols are modeled as finite state machines. An example of a simple building security system, modeled as a finite state machine, etc.

- Many interactive video games are (large, often nondeterministic) finite state machines.

- DNA is the language of life. DNA molecules, as well as the proteins that they describe, are strings that are made up of symbols drawn from small alphabets (nucleotides and amino

acids, respectively). So computational biologists exploit many of the same tools that computational linguists use. For example, they rely on techniques that are based on both finite state machines and context-free grammars.

• Security is perhaps the most important property of many computer systems. The undecidability results show that there cannot exist a general purpose method for automatically verifying arbitrary security properties of programs. The complexity results serve as the basis for powerful encryption techniques.

• Artificial intelligence programs solve problems in task domains ranging from medical diagnosis to factory scheduling. Various logical frameworks have been proposed for representing and reasoning with the knowledge that such programs exploit. The undecidability results that show that there cannot exist a general theorem prover that can decide, given an arbitrary statement in first order logic, whether or not that statement follows from the system's axioms.

Models of computation: purpose and types

Almost any plausible statement about computation is **true** in some model, and **false** in others! A rigorous definition of a model of computation is essential when proving negative results: "impossible...". **Special purpose models** vs. **universal models** of computation (can simulate any other model). ***Algorithm*** and ***computability*** are originally intuitive (describes something or knowing something through feelings rather than conscious reasoning or evidence) concepts. They can remain intuitive as long as we only want to show that some specific result can be computed by following a specific algorithm. Almost always an informal explanation suffices to convince someone with the requisite background that a given algorithm computes a specified result. Everything changes if we wish to show that a desired result is ***not computable*.** The question arises immediately: "What tools are we allowed to use?" The attempt to prove negative results about the nonexistence of certain algorithms forces us to agree on a rigorous definition of *algorithm*.

Mathematics has long investigated problems of the type: what kind of objects can be constructed, what kind of results can be computed using a **limited set of primitives and operations**. For example, the question of what polynomial equations (equation formed with variables, non-negative integer exponents and coefficients, set equal to zero) can be solved using radicals, i.e. using addition, subtraction, multiplication, division, and the extraction of roots, has kept mathematicians busy for centuries.

Whenever the tools allowed are restricted to the extent that “intuitively computable” objects cannot be obtained using these tools alone, we speak of a **special-purpose model of computation**. Such models are of great practical interest because the tools allowed are tailored to the specific characteristics of the objects to be computed, and hence are efficient for this purpose. Many examples are close to hardware design. From the theoretical point of view, however, **universal models of computation** are of prime interest. This concept arose from the natural question "What can be computed by an algorithm, and what cannot?". It was studied during the 1930s by Emil Post (1897–1954), Alonzo Church (1903-1995), Alan Turing (1912–1954), and other logicians. They defined various formal models of computation, such as **production systems, recursive functions, the lambda calculus, and Turing machines,** to capture the intuitive concept of "computation by the application of precise rules". All these different formal models of computation turned out to be equivalent.

This fact greatly strengthens **Church's thesis** that the intuitive concept of algorithm is formalized correctly by any one of these mathematical systems. The models of computation defined by these logicians in the 1930s are **universal** in the sense that they can compute anything computable by any other model, given unbounded resources of time and memory. This concept of universal model of computation is a product of the 20-th century which lies at the center of the theory of computation.

The standard universal models of computation were designed to be conceptually simple: Their primitive operations are chosen to be as weak as possible, as long as they retain their property of being universal computing systems in the sense that they can simulate any computation performed on any other machine. It usually comes as a surprise to novices that the set of primitives of a universal computing machine can be so simple, as long as these machines possess two essential ingredients: *unbounded memory* and *unbounded time*.

Two examples of universal models of computation:

a. **Markov algorithms**, which access data and instructions in a sequential manner, and might be the architecture of choice for computers that have only sequential memory.
b. A **simple random access machines** (RAM), based on a random access memory, whose architecture follows the von Neumann design of stored program computers.

Once one has learned to program basic data manipulation operations, such as moving and copying data and pattern matching, the claim that these primitive "computers" are universal becomes believable. Most simulations of a powerful computer on a simple one share three characteristics:

a. It is straightforward in principle,
b. it involves laborious coding in practice, and
c. it explodes the space and time requirements of a computation.

The weakness of the primitives, desirable from a theoretical point of view, has the consequence that as simple an operation as integer addition becomes an exercise in programming. The purpose of these examples is to support the idea that conceptually simple models of computation are equally powerful, in theory, as models that are much more complex, such as a high-level programming language. Unbounded time and memory is the key that lets the snail ultimately cover the same ground as the hare.

The theory of computability was developed in the 1930s, and greatly expanded in the 1950s and 1960s. Its basic ideas have become part of the foundation that any computer scientist is expected to know. But computability theory is not directly useful. It is based on the concept "computable in principle" but offers no concept of "feasible in practice". And feasibility, rather than "possible in principle", is the touchstone of computer science. Since the 1960s, a theory of the complexity of computation has being developed, with the goal of partitioning the range of computability into complexity classes according to time and space requirements. This theory is still in full development and breaking new ground, in particular with (as yet) exotic models of computation such as quantum computing or DNA computing.

Introduction to formal proof:

Basic Symbols used :

U – Union
∩ - Conjunction
□ - Empty String
Φ – NULL set
7- negation
‘ – compliment
= > implies

Additive inverse: a+(-a)=0

Multiplicative inverse: a*1/a=1

Universal set U={1,2,3,4,5}

Subset A={1,3}

A’ ={2,4,5}

Absorption law: AU(A ∩ B) = A, A ∩ (AUB) = A

De Morgan’s Law:

(AUB)’ =A’ ∩ B’

(A ∩ B)’ = A’ U B’

Double compliment

(A’)’ =A

A ∩ A’ = Φ

Relations:

Let a and b be two sets a relation R contains aXb.

Relations used in TOC:

Reflexive: a = a

Symmetric: aRb = > bRa

Transition: aRb, bRc = > aRc

If a given relation is reflexive, symmentric and transitive then the relation is called equivalence relation.

Deductive proof: Consists of sequence of statements whose truth lead us from some initial statement called the hypothesis or the give statement to a conclusion statement.

The theorem that is proved when we go from a hypothesis H to a conclusion C is the statement "if H then C." We say that C is *deduced* from H. An example

Additional forms of proof:

Proof of sets

Proof by contradiction

Proof by counter example

Direct proof (AKA) Constructive proof:

If *p* is true then *q* is true

E.g.: if a and b are odd numbers then product is also an odd number.

Odd number can be represented as 2n+1

a=2x+1, b=2y+1

product of a X b = (2x+1) X (2y+1)

= 2(2xy+x+y)+1 = 2z+1 (odd number)

Proof by Contrapositive:

easier to prove. The *contrapositive* of the statement "if H then C" is "if not C then not H." A statement and its contrapositive are either both true or both false, so we can prove either to prove the other.

Theorem 1.10: $R \cup (S \cap T) = (R \cup S) \cap (R \cup T)$.

	Statement	Justification
1.	x is in $R \cup (S \cap T)$	Given
2.	x is in R or x is in $S \cap T$	(1) and definition of union
3.	x is in R or x is in both S and T	(2) and definition of intersection
4.	x is in $R \cup S$	(3) and definition of union
5.	x is in $R \cup T$	(3) and definition of union
6.	x is in $(R \cup S) \cap (R \cup T)$	(4), (5), and definition of intersection

Figure 1.5: Steps in the "if" part of Theorem 1.10

	Statement	Justification
1.	x is in $(R \cup S) \cap (R \cup T)$	Given
2.	x is in $R \cup S$	(1) and definition of intersection
3.	x is in $R \cup T$	(1) and definition of intersection
4.	x is in R or x is in both S and T	(2), (3), and reasoning about unions
5.	x is in R or x is in $S \cap T$	(4) and definition of intersection
6.	x is in $R \cup (S \cap T)$	(5) and definition of union

Figure 1.6: Steps in the "only-if" part of Theorem 1.10

To see why "if H then C" and "if not C then not H" are logically equivalent, first observe that there are four cases to consider:

1. H and C both true.

2. H true and C false.

3. C true and H false.

4. H and C both false.

Proof by Contradiction:

H and not C implies falsehood.

That is, start by assuming both the hypothesis H and the negation of the conclusion C. Complete the proof by showing that something known to be false follows logically from H and not C. This form of proof is called *proof by contradiction.*

It often is easier to prove that a statement is not a theorem than to prove it *is* a theorem. As we mentioned, if S is any statement, then the statement "S is not a theorem" is itself a statement without parameters, and thus can be regarded as an observation than a theorem.

Alleged Theorem 1.13: All primes are odd. (More formally, we might say: if integer x is a prime, then x is odd.)

DISPROOF: The integer 2 is a prime, but 2 is even. □

For any sets a,b,c if a ∩ b = Φ and c is a subset of b the prove that a ∩ c = Φ
Given : a ∩ b= Φ and c subset b
Assume: a ∩ c≠ Φ
Then ∀ x, xϵa and xϵc=>xϵb
= > a ∩ b ≠Φ = > a ∩ c= Φ (i.e., the assumption is wrong)

Proof by mathematical Induction:

Suppose we are given a statement $S(n)$, about an integer n, to prove. One common approach is to prove two things:

1. The *basis*, where we show $S(i)$ for a particular integer i. Usually, $i = 0$ or $i = 1$, but there are examples where we want to start at some higher i, perhaps because the statement S is false for a few small integers.
2. The *inductive step*, where we assume $n \geq i$, where i is the basis integer, and we show that "if $S(n)$ then $S(n+1)$."

- ***The Induction Principle***: If we prove $S(i)$ and we prove that for all $n \geq i$, $S(n)$ implies $S(n+1)$, then we may conclude $S(n)$ for all $n \geq i$.

Languages :

The languages we consider for our discussion is an abstraction of natural languages. That is, our focus here is on formal languages that need precise and formal definitions. Programming languages belong to this category.

Symbols :

Symbols are indivisible objects or entity that cannot be defined. That is, symbols are the atoms of the world of languages. A symbol is any single object such as , *a*, 0, 1, #, begin, or do.

Alphabets :

An alphabet is a finite, nonempty set of symbols. The alphabet of a language is normally denoted by Σ. When more than one alphabets are considered for discussion, then subscripts may be used (e.g. $\Sigma_1\Sigma_2$etc) or sometimes other symbol like G may also be

Σ = {0, 1}

Σ = {a, b, c}

Σ = {a, b, c, &, z}

Σ = {#, ▼, ♠, β}

Strings or Words over Alphabet:

A string or word over an alphabet Σ is a finite sequence of concatenated symbols of Σ.

Example: 0110, 11, 001 are three strings over the binary alphabet { 0, 1 } .

aab, abcb, b, cc are four strings over the alphabet { a, b, c }.

It is not the case that a string over some alphabet should contain all the symbols from the alphabet.

For example, the string cc over the alphabet { a, b, c } does not contain the symbols a and b.

Hence, it is true that a string over an alphabet is also a string over any superset of that alphabet.

Length of a string:

The number of symbols in a string w is called its length, denoted by |w|.

Example: | 011 | = 3, |11| = 2, | b | = 1

Some String Operations:

Let x and y be two strings. The concatenation of x and y denoted by xy, is the string. That is, the concatenation of x and y denoted by xy is the string that has a copy of x followed by a copy of y without any intervening space between them.

Example: Consider the string 011 over the binary alphabet. All the prefixes, suffixes and substrings of this string are listed below.

Prefixes: ε, 0, 01, 011.

Suffixes: ε, 1, 11, 011.

Substrings: ε, 0, 1, 01, 11, 011.

Note that x is a prefix (suffix or substring) to x, for any string x and ε is a prefix (suffix or substring) to any string.

A string x is a proper prefix (suffix) of string y if x is a prefix (suffix) of y and $x \neq y$.

In the above example, all prefixes except 011 are proper prefixes.